

JOBSHEET 12
LAPORAN PERCOBAAN PRAKTIKUM
DOUBLE LINKED LIST

Nama : Jonathan Abdiel Haryono
Absen : 14
Kelas : TI 1G

1. Percobaan 1 : Operasi Penambahan pada Double Linked List

Mahasiswa14.java

```
package Jobsheet12;

public class Mahasiswa14 {
    String nim, nama, kelas;
    double ipk;

    Mahasiswa14(String nim, String nama, String kelas, double ipk) {
        this.nim = nim;
        this.nama = nama;
        this.kelas = kelas;
        this.ipk = ipk;
    }

    void tampil() {
        System.out.println(
            "\nNIM      : " + nim +
            "\nNama      : " + nama +
            "\nKelas    : " + kelas +
            "\nIPK       : " + ipk
        );
    }
}
```

Node14.java

```
package Jobsheet12;

public class Node14 {
    Mahasiswa14 data;
    Node14 prev, next;

    Node14(Mahasiswa14 data) {
        this.data = data;
        this.prev = null;
        this.next = null;
    }
}
```

DoubleLinkedList14.java

```
package Jobsheet12;

public class DoubleLinkedList14 {
    Node14 head, tail;

    DoubleLinkedList14() {
        head = null;
        tail = null;
    }

    boolean isEmpty() {
        return (head == null);
    }

    void addFirst(Mahasiswa14 data) {
        Node14 newNode = new Node14(data);

        if (isEmpty()) {
            head = tail = newNode;
        }
        else {
            newNode.next = head;
            head.prev = newNode;
            head = newNode;
        }
    }

    void addLast(Mahasiswa14 data) {
        Node14 newNode = new Node14(data);

        if (isEmpty()) {
            head = tail = newNode;
        }
        else {
            tail.next = newNode;
            newNode.prev = tail;
            tail = newNode;
        }
    }

    void insertAfter(String keyNIM, Mahasiswa14 data) {
        Node14 current = head;

        while (current != null && !current.data.nim.equals(keyNIM)) {
            current = current.next;
        }

        if (current == null) {
            System.out.println("Data dengan NIM " + keyNIM + " tidak ditemukan.");
            return;
        }

        Node14 newNode = new Node14(data);
```

```

        // jika current adalah tail, node baru ditambahkan di akhir
        if (current == tail) {
            newNode.prev = current;
            current.next = newNode;
            tail = newNode;
        }
        else { // node baru disisipkan di tengah
            newNode.prev = current;
            newNode.next = current.next;
            current.next.prev = newNode;
            current.next = newNode;
        }

        System.out.println("Data berhasil disisipkan setelah NIM " + keyNIM);
    }

    void add(Mahasiswa14 data, int index) {

    }

    void print() {
        if (isEmpty()) {
            System.out.println("Linked List masih kosong.");
            return;
        }

        Node14 current = head;
        while (current != null) {
            current.data.tampil();
            current = current.next;
        }
    }

    void removeFirst() {

    }

    void removeLast() {

    }

    void search() { // null ?
    }
}

```

DoubleLinkedListMain14.java

```

package Jobsheet12;
import java.util.Scanner;

public class DoubleLinkedListMain14 {

```

```

static Mahasiswa14 inputMahasiswa() {
    Scanner sc = new Scanner(System.in);

    System.out.println("\nInput Data Mahasiswa");
    System.out.print("Masukkan NIM    : ");
    String nim = sc.nextLine();
    System.out.print("Masukkan Nama    : ");
    String nama = sc.nextLine();
    System.out.print("Masukkan Kelas    : ");
    String kelas = sc.nextLine();
    System.out.print("Masukkan IPK    : ");
    double ipk = sc.nextDouble();

    Mahasiswa14 mhs = new Mahasiswa14(nim, nama, kelas, ipk);
    return mhs;
}

public static void main(String[] args) {
    Scanner scan = new Scanner(System.in);

    DoubleLinkedList14 list = new DoubleLinkedList14();
    int pilihan;

    do {
        System.out.println("\n===== MENU DOUBLE LINKED LIST =====");
        System.out.println("1. Tambah data di awal");
        System.out.println("2. Tambah data di akhir");
        System.out.println("3. Sisipkan data di tengah (setelah NIM)");
        System.out.println("4. Hapus data di awal");
        System.out.println("5. Hapus data di akhir");
        System.out.println("6. Tampilkan data");
        System.out.println("0. Keluar\n");

        System.out.print("Pilih menu : ");
        pilihan = scan.nextInt();
        scan.nextLine();

        switch (pilihan) {
            case 1:
                Mahasiswa14 mhsAwal = inputMahasiswa();
                list.addFirst(mhsAwal);
                break;

            case 2:
                Mahasiswa14 mhsAkhir = inputMahasiswa();
                list.addLast(mhsAkhir);
                break;

            case 3:
                System.out.print("\nMasukkan NIM yang dicari : ");
                String keyNIM = scan.nextLine();

                System.out.print("\nMasukkan data baru :");
                Mahasiswa14 dataBaru = inputMahasiswa();
                list.insertAfter(keyNIM, dataBaru);
                break;
        }
    } while (pilihan != 0);
}

```

```

        case 4:
            list.removeFirst();
            break;

        case 5:
            list.removeLast();
            break;

        case 6:
            list.print();
            break;

        case 0:
            System.out.println("Program selesai.");

        default:
            System.out.println("Menu tidak valid.");
    }
} while (pilihan != 0);
scan.close();
}
}

```

Verifikasi Hasil Percobaan

```

===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar

Masukkan NIM yang dicari : 123005

Masukkan data baru :
Input Data Mahasiswa
Masukkan NIM      : 123010
Masukkan Nama     : Potter
Masukkan Kelas    : 1B
Masukkan IPK      : 3,55
Data berhasil disisipkan setelah NIM 123005

Pilih menu : 2

Input Data Mahasiswa
Masukkan NIM      : 123005
Masukkan Nama     : Harry
Masukkan Kelas    : 1A
Masukkan IPK      : 3,76

===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar

Pilih menu : 6

NIM      : 123005
Nama     : Harry
Kelas   : 1A
IPK      : 3.76

NIM      : 123010
Nama     : Potter
Kelas   : 1B
IPK      : 3.55

===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar

Pilih menu : 3

```

Pertanyaan

1. Jelaskan perbedaan struktur dan mekanisme traversal antara Single Linked List dan Double Linked List!
2. Perhatikan class **Node**, di dalamnya terdapat atribut next dan prev. Jelaskan fungsi masing-masing atribut tersebut pada proses traversal dan manipulasi node!
3. Perhatikan konstruktor pada class **DoubleLinkedList**. Jelaskan fungsi konstruktor tersebut terhadap kondisi awal linked list!
4. Perhatikan potongan kode berikut:

```
if (isEmpty()) {  
    head = tail = newNode;  
}
```

Mengapa head dan tail harus menunjuk node yang sama ketika linked list masih kosong?

5. Modifikasi method **print()** agar menampilkan pesan "Linked List masih kosong" ketika tidak terdapat data pada linked list!
6. Modifikasi kode program dengan menambahkan method **printReverse()** untuk menampilkan seluruh data pada Double Linked List secara terbalik, dimulai dari node tail menuju head!

Jawaban

1. Perbedaan Single Linked List dan Double Linked List :
 - Berdasarkan strukturnya
 - Single Linked List :
 - Tiap node terdiri dari 2 bagian, yakni data (nilai yang disimpan) dan next (pointer ke node selanjutnya).
 - Node terakhir selalu menunjuk ke NULL
 - Double Linked List :
 - Tiap node terdiri dari 3 bagian, yakni prev (pointer yang menunjuk ke node sebelumnya), data (nilai yang disimpan), dan next (pointer yang menunjuk ke node selanjutnya).
 - Node pertama memiliki prev bernilai NULL dan node terakhir memiliki next bernilai NULL.
 - Berdasarkan mekanisme traversalnya (pencarian)
 - Single Linked List :
 - Pencarian bersifat satu arah, dari head (awal) menuju ke belakang sampai kondisi tertentu atau sampai bertemu NULL, yakni next node terakhir.
 - Jika sudah melakukan pencarian hingga node tertentu dan ingin kembali ke node sebelumnya, maka harus mengulang proses pencarian dari awal (head) hingga node yang ingin dicapai.

- Double Linked List :
 - Pencarian bersifat dua arah, dari head ke belakang atau dari tail ke depan.
 - Tak seperti single linked list, double linked list menyimpan node sebelumnya (prev) sehingga dapat berpindah antar node dengan lebih bebas, misalnya jika sedang berada di node tengah, pencarian bisa langsung dilakukan ke depan atau belakang tanpa harus mengulang proses pencarian dari depan seperti single linked list.
- 2. Pada proses traversal, atribut prev dan next digunakan sebagai jembatan dalam program melakukan pencarian atau apapun itu yang memerlukan perpindahan antar node. Atribut prev digunakan sebagai pointer yang menunjuk pada node sebelum node saat ini, sedangkan atribut next digunakan sebagai pointer yang menunjuk pada node setelah node saat ini. Kedua pointer ini berguna dalam program melakukan manipulasi node.

Misalnya, saat program ingin menambahkan data di tengah-tengah dua node tertentu, program akan melakukan pencarian terlebih dahulu (fungsi traversal) berdasarkan data tertentu, misalnya NIM atau nama yang mana tentunya memerlukan atribut next sebagai penghantar menuju node setelahnya dalam melakukan pencarian.

Kemudian setelah ditemukan, atribut prev dan next akan digunakan untuk memanipulasi node (fungsi manipulasi node). Pertama, mengubah prev dari node baru menjadi node saat ini, kemudian mengubah next dari node baru menjadi next dari node saat ini.

Terakhir, kedua node diantara node baru akan disambungkan pada node baru dengan memanipulasi next dan prevnya.

- 3. Pada konstruktor tersebut, ada potongan kode head = null dan tail = null, yakni berfungsi untuk kondisi awal dimana masih belum ada node, sehingga nilai dari head dan tail adalah null (kosong).
- 4. Karena ketika node kosong dan user menambahkan node baru, akan terdapat hanya satu node, sehingga head sekaligus tail dari node tersebut adalah node itu sendiri.
- 5. Telah dicantumkan dalam praktikum, tidak perlu memodifikasi

```
public void print() {  
    if (isEmpty()) {  
        System.out.println(x: "Linked List masih kosong.");  
        return;  
    }  
}
```

```
void print() {  
    if (isEmpty()) {  
        System.out.println(x: "Linked List masih kosong.");  
        return;  
    }  
}
```

```

6. void printReverse() {

    if (isEmpty()) {
        System.out.println("Linked List masih kosong.");
        return;
    }

    Node14 current = tail;
    while (current != null) {
        current.data.tampil();
        current = current.prev;
    }
}

```

Verifikasi modifikasi

<pre> ===== MENU DOUBLE LINKED LIST ===== 1. Tambah data di awal 2. Tambah data di akhir 3. Sisipkan data di tengah (setelah NIM 4. Hapus data di awal 5. Hapus data di akhir 6. Tampilkan data dari awal 7. Tampilkan data dari akhir 0. Keluar Pilih menu : 1 Input Data Mahasiswa Masukkan NIM : 123005 Masukkan Nama : Harry Masukkan Kelas : 1A Masukkan IPK : 3,76 ===== MENU DOUBLE LINKED LIST ===== 1. Tambah data di awal 2. Tambah data di akhir 3. Sisipkan data di tengah (setelah NIM 4. Hapus data di awal 5. Hapus data di akhir 6. Tampilkan data dari awal 7. Tampilkan data dari akhir 0. Keluar Pilih menu : 3 </pre>	<pre> Masukkan NIM yang dicari : 123005 Masukkan data baru : Input Data Mahasiswa Masukkan NIM : 123010 Masukkan Nama : Potter Masukkan Kelas : 1B Masukkan IPK : 3,55 Data berhasil disisipkan setelah NIM 123005 ===== MENU DOUBLE LINKED LIST ===== 1. Tambah data di awal 2. Tambah data di akhir 3. Sisipkan data di tengah (setelah NIM 4. Hapus data di awal 5. Hapus data di akhir 6. Tampilkan data dari awal 7. Tampilkan data dari akhir 0. Keluar Pilih menu : 7 NIM : 123010 Nama : Potter Kelas : 1B IPK : 3.55 NIM : 123005 Nama : Harry Kelas : 1A IPK : 3.76 </pre>
--	--

2. Percobaan 2 : Operasi Penghapusan pada Double Linked List

DoubleLinkedList14.java

```

package Jobsheet12;

public class DoubleLinkedList14 {
    Node14 head, tail;

    DoubleLinkedList14() {
        head = null;
        tail = null;
    }
}

```



```

    }

    boolean isEmpty() {
        return (head == null);
    }

    void addFirst(Mahasiswa14 data) {
        Node14 newNode = new Node14(data);

        if (isEmpty()) {
            head = tail = newNode;
        } else {
            newNode.next = head;
            head.prev = newNode;
            head = newNode;
        }
    }

    void addLast(Mahasiswa14 data) {
        Node14 newNode = new Node14(data);

        if (isEmpty()) {
            head = tail = newNode;
        } else {
            tail.next = newNode;
            newNode.prev = tail;
            tail = newNode;
        }
    }

    void insertAfter(String keyNIM, Mahasiswa14 data) {
        Node14 current = head;

        while (current != null && !current.data.nim.equals(keyNIM)) {
            current = current.next;
        }

        if (current == null) {
            System.out.println("Data dengan NIM " + keyNIM + " tidak ditemukan.");
            return;
        }

        Node14 newNode = new Node14(data);

        // jika current adalah tail, node baru ditambahkan di akhir
        if (current == tail) {
            newNode.prev = current;
            current.next = newNode;
            tail = newNode;
        } else { // node baru disisipkan di tengah
            newNode.prev = current;
            newNode.next = current.next;
            current.next.prev = newNode;
            current.next = newNode;
        }
    }

```

```

        System.out.println("Data berhasil disisipkan setelah NIM " + keyNIM);
    }

    void add(Mahasiswa14 data, int index) {

    }

    void print() {
        if (isEmpty()) {
            System.out.println("Linked List masih kosong.");
            return;
        }

        Node14 current = head;
        while (current != null) {
            current.data.tampil();
            current = current.next;
        }
    }

    void printReverse() {
        if (isEmpty()) {
            System.out.println("Linked List masih kosong.");
            return;
        }

        Node14 current = tail;
        while (current != null) {
            current.data.tampil();
            current = current.prev;
        }
    }

    void removeFirst() {
        if (isEmpty()) {
            System.out.println("Linked List masih kosong.");
            return;
        }

        if (head == tail) {
            head = tail = null;
        } else {
            head = head.next;
            head.prev = null;
        }
    }

    void removeLast() {
        if (isEmpty()) {
            System.out.println("Linked List masih kosong.");
            return;
        }

        if (head == tail) {

```

```

        head = tail = null;
    } else {
        tail = tail.prev;
        tail.next = null;
    }
}
}
}

```

DoubleLinkedList14.java

```
package Jobsheet12;
```

```
import java.util.Scanner;
```

```
public class DoubleLinkedListMain14 {
```

```

    static Mahasiswa14 inputMahasiswa() {
        Scanner sc = new Scanner(System.in);

```

```

        System.out.println("\nInput Data Mahasiswa");
        System.out.print("Masukkan NIM    : ");
        String nim = sc.nextLine();
        System.out.print("Masukkan Nama   : ");
        String nama = sc.nextLine();
        System.out.print("Masukkan Kelas  : ");
        String kelas = sc.nextLine();
        System.out.print("Masukkan IPK    : ");
        double ipk = sc.nextDouble();

```

```

        Mahasiswa14 mhs = new Mahasiswa14(nim, nama, kelas, ipk);
        return mhs;
    }

```

```

    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);

```

```

        DoubleLinkedList14 list = new DoubleLinkedList14();
        int pilihan;

```

```

        do {
            System.out.println("\n===== MENU DOUBLE LINKED LIST =====");
            System.out.println("1. Tambah data di awal");
            System.out.println("2. Tambah data di akhir");
            System.out.println("3. Sisipkan data di tengah (setelah NIM)");
            System.out.println("4. Hapus data di awal");
            System.out.println("5. Hapus data di akhir");
            System.out.println("6. Tampilkan data dari awal");
            System.out.println("7. Tampilkan data dari akhir");
            System.out.println("0. Keluar\n");

```

```

            System.out.print("Pilih menu : ");
            pilihan = scan.nextInt();
            scan.nextLine();

```

```

            switch (pilihan) {
                case 1:
                    Mahasiswa14 mhsAwal = inputMahasiswa();
                    list.addFirst(mhsAwal);

```

```

        break;

    case 2:
        Mahasiswa14 mhsAkhir = inputMahasiswa();
        list.addLast(mhsAkhir);
        break;

    case 3:
        System.out.print("\nMasukkan NIM yang dicari : ");
        String keyNIM = scan.nextLine();

        System.out.print("\nMasukkan data baru :");
        Mahasiswa14 dataBaru = inputMahasiswa();
        list.insertAfter(keyNIM, dataBaru);
        break;

    case 4:
        list.removeFirst();
        break;

    case 5:
        list.removeLast();
        break;

    case 6:
        list.print();
        break;

    case 7:
        list.printReverse();
        break;

    case 0:
        System.out.println("Program selesai.");

    default:
        System.out.println("Menu tidak valid.");
    }
} while (pilihan != 0);
scan.close();
}
}

```

Verifikasi Hasil Percobaan

```
===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data dari awal
7. Tampilkan data dari akhir
0. Keluar

Pilih menu : 5

===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data dari awal
7. Tampilkan data dari akhir
0. Keluar

Pilih menu : 6

NIM      : 123005
Nama     : Harry
Kelas   : 1A
IPK      : 3.76
```

Pertanyaan

1. Perhatikan potongan kode berikut pada method `removeFirst()`:

```
head = head.next;
head.prev = null;
```

Jelaskan fungsi masing-masing statement tersebut pada proses penghapusan node!

2. Modifikasi method `removeFirst()` dan `removeLast()` agar program menampilkan data yang berhasil dihapus!

Jawaban

1. Statement pertama, `head = head.next`, mengubah head baru menjadi next dari head, yakni menjadikan node setelah head sebagai head yang baru karena head yang lama akan dihapus. Statement kedua, `head.prev = null`, menjadikan prev dari head yang baru sebagai null, artinya menghapus head yang lama dengan cara menjadikannya bernilai null.
2. Perubahan kode pada `DoubleLinkedList14.java`
`package Jobsheet12;`

```
public class DoubleLinkedList14 {
    Node14 head, tail;
```

```

DoubleLinkedList14() {
    head = null;
    tail = null;
}

boolean isEmpty() {
    return (head == null);
}

void addFirst(Mahasiswa14 data) {
    Node14 newNode = new Node14(data);

    if (isEmpty()) {
        head = tail = newNode;
    }
    else {
        newNode.next = head;
        head.prev = newNode;
        head = newNode;
    }
}

void addLast(Mahasiswa14 data) {
    Node14 newNode = new Node14(data);

    if (isEmpty()) {
        head = tail = newNode;
    }
    else {
        tail.next = newNode;
        newNode.prev = tail;
        tail = newNode;
    }
}

void insertAfter(String keyNIM, Mahasiswa14 data) {
    Node14 current = head;

    while (current != null && !current.data.nim.equals(keyNIM)) {
        current = current.next;
    }

    if (current == null) {
        System.out.println("Data dengan NIM " + keyNIM + " tidak ditemukan.");
        return;
    }

    Node14 newNode = new Node14(data);

    // jika current adalah tail, node baru ditambahkan di akhir
    if (current == tail) {
        newNode.prev = current;
        current.next = newNode;
        tail = newNode;
    }
}

```

```

    }
    else { // node baru disisipkan di tengah
        newNode.prev = current;
        newNode.next = current.next;
        current.next.prev = newNode;
        current.next = newNode;
    }

    System.out.println("Data berhasil disisipkan setelah NIM " + keyNIM);
}

void print() {
    if (isEmpty()) {
        System.out.println("Linked List masih kosong.");
        return;
    }

    Node14 current = head;
    while (current != null) {
        current.data.tampil();
        current = current.next;
    }
}

void printReverse() {
    if (isEmpty()) {
        System.out.println("Linked List masih kosong.");
        return;
    }

    Node14 current = tail;
    while (current != null) {
        current.data.tampil();
        current = current.prev;
    }
}

void removeFirst() {
    if (isEmpty()) {
        System.out.println("Linked List masih kosong.");
        return;
    }

    if (head == tail) {
        head = tail = null;
    }
    else {
        head = head.next;

        System.out.println("\nData berhasil dihapus");
        printOne(head.prev);

        head.prev = null;
    }
}

```

```

void removeLast() {
    if (isEmpty()) {
        System.out.println("Linked List masih kosong.");
        return;
    }

    if (head == tail) {
        head = tail = null;
    }
    else {
        tail = tail.prev;

        System.out.println("\nData berhasil dihapus");
        printOne(tail.next);

        tail.next = null;
    }
}

void printOne(Node14 node) {
    System.out.println(
        "\nNIM      : " + node.data.nim +
        "\nNama      : " + node.data.nama +
        "\nKelas     : " + node.data.kelas +
        "\nIPK       : " + node.data.ipk
    );
}
}

```

Verifikasi hasil :

```

===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data dari awal
7. Tampilkan data dari akhir
0. Keluar

Pilih menu : 5

Data berhasil dihapus

NIM      : 123010
Nama     : Potter
Kelas   : 1B
IPK      : 3.55

===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data dari awal
7. Tampilkan data dari akhir
0. Keluar

Pilih menu : 6

NIM      : 123005
Nama     : Harry
Kelas   : 1A
IPK      : 3.76

```